NeoFlow: A Flexible Framework for Enabling Efficient Compilation for High Performance DNN Training

Size Zheng, *Student Member, IEEE,* Renze Chen, Yicheng Jin, Anjiang Wei, Bingyang Wu, Xiuhong Li, Shengen Yan, Yun Liang, *Senior Fellow, IEEE,*

Abstract—Deep neural networks (DNNs) are increasingly deployed in various image recognition and natural language processing applications. The continuous demand for accuracy and high performance has led to innovations in DNN design and a proliferation of new operators. However, existing DNN training frameworks such as PyTorch and TensorFlow only support a limited range of operators and rely on hand-optimized libraries to provide efficient implementations for these operators. To evaluate novel neural networks with new operators, the programmers have to either replace the holistic new operators with existing operators or provide low-level implementations manually. Therefore, a critical requirement for DNN training frameworks is to provide high-performance implementations for the neural networks containing new operators automatically in the absence of efficient library support.
In this paper, we introduce NeoFlow, which is a flexible framework for enabling efficient compilation for high-performance DNN training. NeoFlow allows the programmers to directly write customized expressions as new operators to be mapped to graph representation and low-level implementations automatically, providing both high programming productivity and high performance. First, NeoFlow provides expression-based automatic differentiation to support customized model definitions with new operators. Then, NeoFlow proposes an efficient compilation system that partitions the neural network graph into subgraphs, explores optimized schedules, and generates high-performance libraries for subgraphs automatically. Finally, NeoFlow develops an efficient runtime system to combine the compilation and training as a whole by overlapping their execution. In the experiments, we examine the numerical accuracy and performance of NeoFlow. The results show that NeoFlow can achieve similar or even better performance at the operator and whole graph level for DNNs compared to deep learning frameworks. Especially, for novel networks training, the geometric mean speedups of NeoFlow to PyTorch, TensorFlow, and CuDNN are 3.16X, 2.43X, and 1.92X, respectively.

Index Terms—Deep Learning, Training, Code Generation, Compiler Optimization, Automatic Differentiation

1 Introduction

Deep neural networks (DNNs) have gained great breakthroughs in various domains including image processing [1], [2], [3] and natural language processing [4], [5], [6]. The excellent performance of these DNNs often comes with a long training time. Commonly, training a model may take hours or even days. Today, the best practice of training DNN models is to use frameworks such as PyTorch [7], TensorFlow [8], and MxNet [9]. The users implement their DNN models in these frameworks by defining the DNN graph, In general, a DNN graph is composed of a series of tensors and operators, where a tensor is regarded as a multidimensional array and an operator is a function applied to the tensors (e.g., convolution). After that, the deep learning frameworks rely on high-performance hand-optimized libraries to accelerate the execution of DNN graphs. For example, on Nvidia GPUs, PyTorch and TensorFlow leverage CuDNN library [10] to accelerate DNN operators such as convolution and batch normalization by mapping them to the corresponding APIs of CuDNN.

The dramatic growth in use has bolstered new DNN models such as CapsuleNet [11], [12], [13], MI-LSTM [14], and SC-RNN [15]. However, existing DNN training frameworks heavily rely on hand-optimized libraries and thus lack support for these emerging DNN models with new operators. For example, there is currently no implementation of capsule convolution [11], [12], [13] in CuDNN, and as a result, deep learning frameworks can't provide an efficient implementation for this operator. There are two possible approaches to deal with this problem. One approach is to use existing operators to assemble a computation graph that is functionally equivalent to the new operator. For example, the users can use several 2D convolutions to substitute a capsule convolution by assembling the results of these convolutions. However, this approach breaks a holistic operator into small operators and introduces additional tensor transformation overhead such as slicing and reshaping, resulting in a complex implementation and low performance. More importantly, this manual process requires significant effort and scales poorly as the number of new operators increases. Another approach is to implement the low-level kernels (e.g., CUDA kernels) manually for the new operator and register them into the deep learning frameworks. But the low-level kernel programming and optimization is time-consuming and error-prone. A less optimized implementa-

- *Size Zheng, Renze Chen, Yicheng Jin, Anjiang Wei, Bingyang Wu, and Yun Liang are with the Department of Computer Science, School of EECS, Peking University, Beijing, China. E-mail: {zhengsz, crz, yicheng.jin weianjiang, bingyangwu, ericlyun}@pku.edu.cn*
- *Xiuhong Li and Shengen Yan are with SenseTime Research. E-mail: {lixiuhong, yanshengen}@sensetime.com*

(Corresponding author: Yun Liang.)